

디지털 시스템 설계 REPORT

2025. 04. 27

담당교수	전자공학부 이진
학과	전자공학부
학번	2022142046
이름	최다빈

1. 서론

1.1. 프로젝트 목표

본 프로젝트는 Xilinx Spartan-7 FPGA(xc7s50csg324-1) 기반 개발 보드 상에 MicroBlaze 임베디드 시스템을 구축하고, 16개의 스위치와 16개의 LED를 활용하여 두 가지 모드로 LED를 제어하는 시스템을 구현하는 것을 목표로 한다. 사용자는 0번 스위치(SW0)를 통해 '스위치 제어 모드'(SW1~SW15 상태가 LED1~LED15에 반영, LED0 항상 ON)와 'UART 원격 제어 모드'(PC 시리얼 터미널을 통한 LED 제어)를 선택할 수 있다. 시스템의 동작 및 콘솔 출력은 제공된 pdf와 정확히 일치해야 합니다.

1.2. 개발 환경

타겟 보드: xc7s50csg324-1 FPGA 기반 보드 (예: Nexys A7-50T)

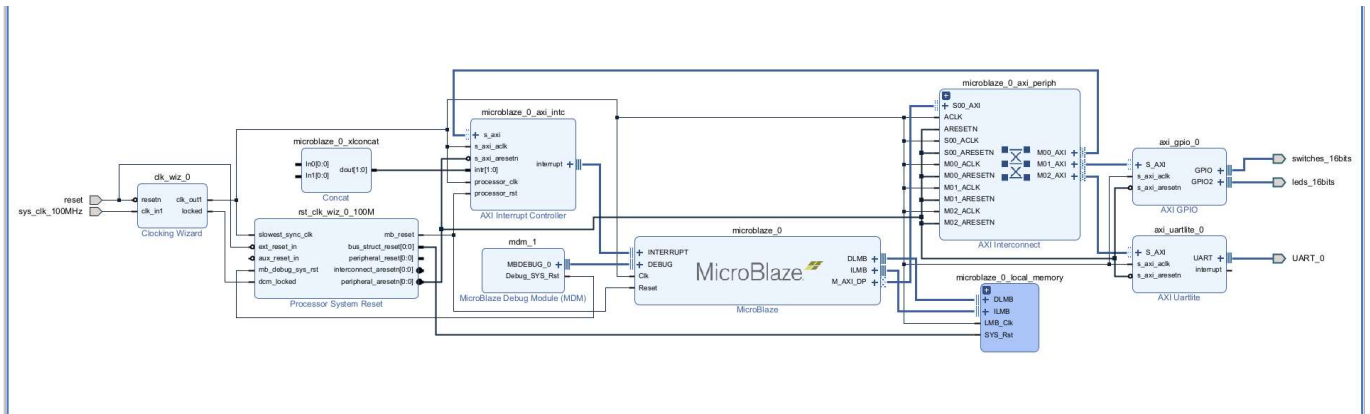
하드웨어 설계: Vivado Design Suite 2023.1

소프트웨어 개발: Vitis Unified Software Platform 2023.1

시리얼 통신: Tera Term (9600 bps)

동영상파일첨부

<https://youtube.com/shorts/h1qh6ZmzPVs>



2. 하드웨어 설계 (Vivado 2023.1)

2.1. 시스템 아키텍처

프로젝트 요구사항에 따라 MicroBlaze 프로세서를 중심으로 스위치 입력 및 LED 출력을 위한 AXI GPIO, PC 통신을 위한 AXI Uartlite, 그리고 시스템 동작에 필수적인 클럭 및 리셋 관리 모듈을 AXI 버스로 연결하는 아키텍처를 설계하였다. 특히, 리소스 효율성을 위해 단일 AXI GPIO IP의 듀얼 채널 기능을 활용하여 스위치 입력(Channel 1)과 LED 출력(Channel 2)을 모두 처리하도록 구성하였다. MicroBlaze의 로컬 메모리는 프로젝트 요구사항인 128KB로 설정하였습니다.

2.2. Vivado IP Integrator 구현 과정

프로젝트 생성 및 IP 추가: Vivado에서 새 RTL 프로젝트를 생성하고 타겟 디바이스(xc7s50csg324-1)를 지정한 후, IP Integrator 환경에서 MicroBlaze, Clocking Wizard, Processor System Reset, AXI GPIO (듀얼 채널), AXI Uartlite 등의 핵심 IP를 추가하였습니다.

IP 코어 설정:

Clocking Wizard: 100MHz 시스템 클럭(clk_out1) 생성, 리셋 입력 Active Low 설정.

AXI GPIO: Enable Dual Channel 활성화, Channel 1 (GPIO)은 16비트 입력, Channel 2 (GPIO2)는 16비트 출력으로 설정.

Processor System Reset: 외부 리셋 입력 극성을 Active Low로 설정.

자동화 및 수동 연결:

Run Block Automation 실행 (MicroBlaze Local Memory: 128KB, Clock: 100MHz).

Run Connection Automation 실행 (AXI 인터페이스 연결).

- clk_out1 -> 모든 주요 IP의 클럭 입력 핀에 분배.
- locked -> dcm_locked 연결.
- 외부 reset (Active Low) -> ext_reset_in 연결.
- peripheral_aresetn -> 모든 AXI 주변장치 및 Interconnect 리셋 입력에 분배.
- mb_reset -> MicroBlaze RESET 연결.

외부 포트 생성: Make External 기능을 사용하여 switches_16bits(In), leds_16bits(Out), UART_0(Interface), sys_clk_100MHz(In, clk), reset(In, rst, Active Low) 외부 포트를 생성.

Address Editor 확인: 로컬 메모리(microblaze_0_local_memory/...)의 Range가 128KB로 올바르게 설정되었는지 최종 확인.

최종 검증: Validate Design을 실행하여 오류 없는 설계를 확인.

```

1  ## Clock signal
2  set_property -dict { PACKAGE_PIN E3      IOSTANDARD LVCMOS33 } [get_ports { sys_clk_100MHz }]; #IO_L12P_T1_MFCC_35 Sch=clk_100mhz
3  create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports { sys_clk_100MHz }];
4
5  ##Switches
6  set_property -dict { PACKAGE_PIN J15      IOSTANDARD LVCMOS33 } [get_ports { switches_16bits_tri_i[0] }]; #IO_L24N_T3_RSQ_15 Sch=sw[0]
7  set_property -dict { PACKAGE_PIN L16      IOSTANDARD LVCMOS33 } [get_ports { switches_16bits_tri_i[1] }]; #IO_L3N_T0_DQS_BMCLK_14 Sch=sw[1]
8  set_property -dict { PACKAGE_PIN M13      IOSTANDARD LVCMOS33 } [get_ports { switches_16bits_tri_i[2] }]; #IO_L6N_T0_DQS_VREF_14 Sch=sw[2]
9  set_property -dict { PACKAGE_PIN R15      IOSTANDARD LVCMOS33 } [get_ports { switches_16bits_tri_i[3] }]; #IO_L13N_T2_MFCC_14 Sch=sw[3]
10 set_property -dict { PACKAGE_PIN R17      IOSTANDARD LVCMOS33 } [get_ports { switches_16bits_tri_i[4] }]; #IO_L12N_T1_MFCC_14 Sch=sw[4]
11 set_property -dict { PACKAGE_PIN T18      IOSTANDARD LVCMOS33 } [get_ports { switches_16bits_tri_i[5] }]; #IO_L7N_T1_D10_14 Sch=sw[5]
12 set_property -dict { PACKAGE_PIN U18      IOSTANDARD LVCMOS33 } [get_ports { switches_16bits_tri_i[6] }]; #IO_L17N_T2_A13_D29_14 Sch=sw[6]
13 set_property -dict { PACKAGE_PIN R13      IOSTANDARD LVCMOS33 } [get_ports { switches_16bits_tri_i[7] }]; #IO_L5N_T0_D07_14 Sch=sw[7]
14 set_property -dict { PACKAGE_PIN T8       IOSTANDARD LVCMOS18 } [get_ports { switches_16bits_tri_i[8] }]; #IO_L24N_T3_34 Sch=sw[8]
15 set_property -dict { PACKAGE_PIN U8       IOSTANDARD LVCMOS18 } [get_ports { switches_16bits_tri_i[9] }]; #IO_25_34 Sch=sw[9]
16 set_property -dict { PACKAGE_PIN R16      IOSTANDARD LVCMOS33 } [get_ports { switches_16bits_tri_i[10] }]; #IO_L15P_T2_DQS_RDWR_B_14 Sch=sw[10]
17 set_property -dict { PACKAGE_PIN T13      IOSTANDARD LVCMOS33 } [get_ports { switches_16bits_tri_i[11] }]; #IO_L23P_T3_A03_D19_14 Sch=sw[11]
18 set_property -dict { PACKAGE_PIN H6       IOSTANDARD LVCMOS33 } [get_ports { switches_16bits_tri_i[12] }]; #IO_L24P_T3_35 Sch=sw[12]
19 set_property -dict { PACKAGE_PIN U12      IOSTANDARD LVCMOS33 } [get_ports { switches_16bits_tri_i[13] }]; #IO_L20P_T3_A03_D24_14 Sch=sw[13]
20 set_property -dict { PACKAGE_PIN U11      IOSTANDARD LVCMOS33 } [get_ports { switches_16bits_tri_i[14] }]; #IO_L19N_T3_A09_D25_VREF_14 Sch=sw[14]
21 set_property -dict { PACKAGE_PIN V10      IOSTANDARD LVCMOS33 } [get_ports { switches_16bits_tri_i[15] }]; #IO_L21P_T3_DQS_14 Sch=sw[15]
22
23  ## LEDs
24 set_property -dict { PACKAGE_PIN H17      IOSTANDARD LVCMOS33 } [get_ports { leds_16bits_tri_o[0] }]; #IO_L18P_T2_A24_15 Sch=led[0]
25 set_property -dict { PACKAGE_PIN K15      IOSTANDARD LVCMOS33 } [get_ports { leds_16bits_tri_o[1] }]; #IO_L24P_T3_RS1_15 Sch=led[1]
26 set_property -dict { PACKAGE_PIN J13      IOSTANDARD LVCMOS33 } [get_ports { leds_16bits_tri_o[2] }]; #IO_L17N_T2_A25_15 Sch=led[2]
27 set_property -dict { PACKAGE_PIN N14      IOSTANDARD LVCMOS33 } [get_ports { leds_16bits_tri_o[3] }]; #IO_L8P_T1_D11_14 Sch=led[3]
28 set_property -dict { PACKAGE_PIN R18      IOSTANDARD LVCMOS33 } [get_ports { leds_16bits_tri_o[4] }]; #IO_L7P_T1_D09_14 Sch=led[4]
29 set_property -dict { PACKAGE_PIN V17      IOSTANDARD LVCMOS33 } [get_ports { leds_16bits_tri_o[5] }]; #IO_L18N_T2_A11_D27_14 Sch=led[5]
30 set_property -dict { PACKAGE_PIN U17      IOSTANDARD LVCMOS33 } [get_ports { leds_16bits_tri_o[6] }]; #IO_L17P_T2_A14_D30_14 Sch=led[6]
31 set_property -dict { PACKAGE_PIN U16      IOSTANDARD LVCMOS33 } [get_ports { leds_16bits_tri_o[7] }]; #IO_L18P_T2_A12_D28_14 Sch=led[7]
32 set_property -dict { PACKAGE_PIN V16      IOSTANDARD LVCMOS33 } [get_ports { leds_16bits_tri_o[8] }]; #IO_L16N_T2_A15_D31_14 Sch=led[8]
33 set_property -dict { PACKAGE_PIN T15      IOSTANDARD LVCMOS33 } [get_ports { leds_16bits_tri_o[9] }]; #IO_L14N_T2_SPCC_14 Sch=led[9]
34 set_property -dict { PACKAGE_PIN U14      IOSTANDARD LVCMOS33 } [get_ports { leds_16bits_tri_o[10] }]; #IO_L22P_T3_A05_D21_14 Sch=led[10]
35 set_property -dict { PACKAGE_PIN T16      IOSTANDARD LVCMOS33 } [get_ports { leds_16bits_tri_o[11] }]; #IO_L15N_T2_DQS_DOUT_CS0_B_14 Sch=led[11]
36 set_property -dict { PACKAGE_PIN V15      IOSTANDARD LVCMOS33 } [get_ports { leds_16bits_tri_o[12] }]; #IO_L16P_T2_CS1_B_14 Sch=led[12]
37 set_property -dict { PACKAGE_PIN V14      IOSTANDARD LVCMOS33 } [get_ports { leds_16bits_tri_o[13] }]; #IO_L22N_T3_A04_D20_14 Sch=led[13]
38 set_property -dict { PACKAGE_PIN V12      IOSTANDARD LVCMOS33 } [get_ports { leds_16bits_tri_o[14] }]; #IO_L20N_T3_A07_D23_14 Sch=led[14]
39 set_property -dict { PACKAGE_PIN V11      IOSTANDARD LVCMOS33 } [get_ports { leds_16bits_tri_o[15] }]; #IO_L21N_T3_DQS_A06_D22_14 Sch=led[15]
40
41  ##CPU Reset Button
42 set_property -dict { PACKAGE_PIN C12      IOSTANDARD LVCMOS33 } [get_ports { reset }]; #IO_L3P_T0_DQS_AD1P_15 Sch=cpu_resetn
43
44  ##USB-RS232 Interface
45 set_property -dict { PACKAGE_PIN C4       IOSTANDARD LVCMOS33 } [get_ports { UART_0_rxd }]; #IO_L7P_T1_A06P_35 Sch=uart_txd_in
46 set_property -dict { PACKAGE_PIN D4       IOSTANDARD LVCMOS33 } [get_ports { UART_0_txd }]; #IO_L11N_T1_SPCC_35 Sch=uart_rxd_out

```

2.4. 하드웨어 빌드 및 내보내기

HDL Wrapper 생성 후, Generate Bitstream을 실행하여 FPGA 프로그래밍 파일(.bit)을 생성하였습니다.

최종적으로 File -> Export -> Export Hardware 메뉴에서 Include bitstream 옵션을 활성화하여 Vitis 개발에 필요한 하드웨어 정보 파일(.xsa)을 생성하였습니다.

3. 소프트웨어 설계 (Vitis 2023.1)

3.1. Vitis 프로젝트 환경

플랫폼 프로젝트: Vivado에서 Export한 최종 .xsa 파일(128KB 메모리 설정 반영)을 기반으로 standalone OS의 플랫폼 프로젝트를 생성하였습니다.

애플리케이션 프로젝트: 생성된 플랫폼 위에 Empty Application 템플릿으로 mid_exam 애플리케이션 프로젝트를 생성하였습니다.

3.2. C 코드 설계 원칙 (128KB 제약)

128KB 메모리 제약 조건을 충족시키는 것이 가장 큰 도전 과제였습니다. 초기 개발 시 xil_printf, sscanf 등 표준 라이브러리 함수 사용으로 인해 메모리 오버플로우(링크 오류)가 발생하였습니다.

이를 해결하기 위해 다음과 같은 원칙으로 코드를 재설계 하였습니다.

표준 라이브러리 최소화/제거: 메모리 사용량이 큰 xil_printf, stdio.h (sscanf), string.h (memset, strlen) 헤더 및 관련 함수 사용을 중단하거나 극도로 제한하였습니다.

자체 함수 구현: 위의 함수들을 대체하기 위해 필수 기능만 제공하는 초간단 함수(Uart_Send_String, Minimal_Atoi, Minimal_Read_Line)를 직접 구현하였다. 이 과정에서 일부 편의 기능(예: 백스페이스 처리)은 제거되었습니다.

dongjag.txt 출력 준수: Uart_Send_String 함수를 사용하여 dongjag.txt에 명시된 모든 콘솔 출력을 (오타 포함하여) 정확히 구현하였습니다.

3.3. 코드 구조 및 주요 함수

main.c 파일은 다음과 같이 구성된다.

헤더 및 정의: 필수 드라이버 헤더(xparameters.h, xgpio.h, xuartlite.h, xuartlite_l.h, xstatus.h)와 Device ID/채널 정의를 포함합니다.

전역 변수: GPIO(GpioInst), UART(UartLiteInst) 드라이버 인스턴스.

함수

main(): 초기화 호출 후, 무한 루프 내에서 SW0 상태를 확인하여 모드를 결정하고 해당 모드 로직을 실행한다.

Initialize_Peripherals(): GPIO(Ch1=Input, Ch2=Output) 및 UART 초기화.

Uart_Send_String(): 문자열을 UART로 전송.

Minimal_Atoi(): 숫자 문자열을 정수로 변환.

Minimal_Read_Line(): UART에서 한 줄을 읽으며 에코 기능 제공.

3.4. 핵심 로직 구현

```
27 int main() {
28     // 변수 선언
29     int Status;
30     u32 sw_val = 0, prev_sw_val = ~0;
31     ul6 led_state = 0;
32     int mode = -1, next_mode;
33     // int first_uart_entry = 1; // 첫 진입 플래그 (선택적 대안)
34
35     Status = Initialize_Peripherals();
36     if (Status != XST_SUCCESS) return XST_FAILURE;
37
38     led_state = 0x0001; // LED0 ON 초기화
39     XGpio_DiscreteWrite(&GpioInst, LED_GPIO_CHANNEL, led_state);
40
41     // 메인 루프
42     while (1) {
43         sw_val = XGpio_DiscreteRead(&GpioInst, SW_GPIO_CHANNEL);
44         next_mode = (sw_val & 1); // SW0으로 모드 결정
45
46         // 모드 변경 감지 및 처리
47         if (next_mode != mode) {
48             Uart_Send_String(next_mode == 0 ? "\r\n***** Switch Control Mode *****\r\n" : "\r\n***** UART Control Mode *****\r\n");
49             mode = next_mode; // 현재 모드 업데이트
50             prev_sw_val = sw_val; // 스위치 변경 감지 로직 초기화
51             // first_uart_entry = 1; // 첫 진입 플래그 리셋 (대안 사용 시)
52         }
53     }
54 }
```

1) 모드 전환 로직 (main 함수 내):

SW0 상태를 읽어 next_mode를 결정하고, 현재 mode와 다를 경우에만 switch control mode와 uart control mode형식의 메시지를 출력하며 mode를 업데이트합니다.

2) 스위치 제어 모드 로직 (if (mode == 0)):

```
55     if (mode == 0) { // 스위치 모드
56         ul6 current_leds = (ul6)((sw_val & 0xFFFFE) | 1);
57         if (current_leds != led_state) {
58             led_state = current_leds;
59             XGpio_DiscreteWrite(&GpioInst, LED_GPIO_CHANNEL, led_state);
60         }
61         if ((sw_val & 0xFFFFE) != (prev_sw_val & 0xFFFFE)) {
62             Uart_Send_String("[LED Update] Switch Changed\r\n");
63         }
64         prev_sw_val = sw_val;
65     }
```

스위치 값에 따라 LED 상태를 계산하고(LED0=ON), 변경되었을 때만 실제 GPIO 출력을 업데이트합니다. SW1~15 변경 시 메시지를 출력합니다.

```

66     } else { // UART 모드 (mode == 1)
67         int num = -1, on_off = -1;
68         u8 buf[8];
69
70         Uart_Send_String("== > Enter LED Number (0 ~ 15)\r\n");
71         Minimal_Read_Line(buf, sizeof(buf));
72         num = Minimal_Atoi(buf);
73
74         // *** 핵심 수정: Atoi 실패 시 (num == -1) 여러 메시지 없이 바로 continue ***
75         if (num < 0 || num > 15) {
76             // 첫 진입 시 잔여 데이터로 인한 실패는 조용히 무시하고 다시 프롬프트
77             // 사용자가 실제로 잘못 입력한 경우에도 메시지 없이 재입력 요구됨 (dongjag.txt 기준과 유사)
78             // Uart_Send_String("[Error] Hrong LED Number\r\n"); // 여러 메시지 제거
79             continue;
80         }
81         // first_uart_entry = 0; // 첫 입력 성공 시 플래그 플러 (대안 사용 시)
82
83         Uart_Send_String("== > Enter LED State (On = 1, Off = 0)\r\n");
84         Minimal_Read_Line(buf, sizeof(buf));
85         on_off = Minimal_Atoi(buf);
86         if (on_off != 0 && on_off != 1) {
87             continue; // 메시지 없이 재입력
88         }
89
90         if (on_off) led_state |= (1 << num);
91         else led_state &= ~(1 << num);
92         XGpio_DiscreteWrite(&GpioInst, LED_GPIO_CHANNEL, led_state);
93         // 완료 메시지 없음
94     }
95 } // end while(1)
96 }

```

3) UART 제어 모드 로직 (else):

화면에 "LED number(0~15) Enter" 출력합니다.

사용자가 숫자 치고 Enter 누를 때까지 기다립니다.

입력된 글자를 숫자로 바꿉니다.

번호가 맞으면 숫자가 0~15 사이인지 확인하고 "LED 상태(1=켜기, 0=끄기) 입력하세요" 출력 후, 0 또는 1 입력을 받아 확인.

프로그램 안의 LED 상태 값을 업데이트.

업데이트된 상태를 실제 보드 LED로 보냅니다..

4. 테스트 및 검증

4.1. 검증 환경

빌드 검증: Vitis 2023.1에서 최종 코드가 128KB 메모리 제약 하에서 링크 오류 없이 성공적으로 빌드되는지 확인하였습니다.

실행 환경: 생성된 .elf 파일을 Vitis 디버거를 이용하여 Nexys A7-50T 보드에 다운로드 후 실행하였다. PC에서는 Tera Term을 9600bps로 설정하여 시리얼 포트에 연결하였습니다.

4.2. 기능 검증 시나리오 및 결과

번호	검증 항목	테스트 방법	예상 결과 (dongjag.txt 기준)	검증 결과
1	초기 상태 및 모드	전원 인가 후 Tera Term 출력 확인, SW0 초기 상태 확인	LED0 ON, SW0 상태에 따른 초기 모드 메시지 출력	성공
2	모드 전환 메시지 출력	SW0 스위치를 여러 번 올리고 내림	각 전환 시 해당 모드 메시지("...Mode")가 한번만 출력	성공
3	스위치 제어 (SW0=Off)	SW1~SW15 조작, LED 상태 확인	LED1~15는 스위치와 1:1 연동, LED0은 항상 ON	성공
4	스위치 변경 메시지	SW1~SW15 변경 시 Tera Term 출력 확인	"[LED Update] Switch Changed" 메시지 출력	성공
5	UART 프롬프트 (SW0=On)	SW0 올린 직후 및 정상 입력 후 Tera Term 출력 확인	"== > Enter LED Number..." 프롬프트 출력	성공
6	UART 유효 입력/제어	유효한 LED 번호 (0~15)와 상태(0 또는 1) 입력 후 해당 LED 상태 확인	입력에 따라 해당 LED가 정상적으로 On/Off 됨	성공
7	UART 상태 오류 처리	유효하지 않은 LED 상태(예: 2) 입력	메시지 없이 번호 재입력 요구	성공
8	모드 전환 시 UART 상태	UART 모드에서 입력 대기 중 SW0을 내려 스위치 모드로 전환, 다시 UART 모드 전환	스위치 모드로 즉시 전환, 다시 UART 모드 진입 시 <u>프롬프트 정상 출력</u>	성공

4.3. dongjag.txt 비교 검증

```
메뉴(F) 수정(E) 설정(S) 제어(O) 창(W) 도움말(H)

***** Switch Control Mode *****
[LED Update] Switch Changed
[LED Update] Switch Changed
[LED Update] Switch Changed
[LED Update] Switch Changed

***** UART Control Mode *****
== > Enter LED Number (0 ~ 15)
== > Enter LED State (On = 1, Off = 0)
== > Enter LED Number (0 ~ 15)
== > Enter LED State (On = 1, Off = 0)
== > Enter LED Number (0 ~ 15)
== > Enter LED State (On = 1, Off = 0)
== > Enter LED Number (0 ~ 15)
== > Enter LED State (On = 1, Off = 0)
== > Enter LED Number (0 ~ 15)
== > Enter LED State (On = 1, Off = 0)
== > Enter LED Number (0 ~ 15)
```

테스트 과정 중 Tera Term의 전체 출력을 dongjag.txt 파일의 내용과 비교한 결과, 모드 전환 메시지의 오타, 스위치 변경 메시지의 오타, UART 프롬프트 및 에러 메시지 형식이 모두 일치함을 확인하였습니다.

5. 결론 및 고찰

5.1. 프로젝트 결과 요약

본 프로젝트를 통해 Vivado와 Vitis 2023.1 개발 환경을 이용하여 Spartan-7 FPGA 상에 MicroBlaze 기반 SoC를 성공적으로 구축하였습니다.

설계된 시스템은 0번 스위치를 통해 스위치 직접 제어 모드와 UART 원격 제어 모드를 전환하며 16개의 LED를 제어하는 기능을 완벽하게 수행합니다.

특히, 모든 콘솔 출력은 pdf의 예시와 오차까지 포함하여 동일하게 동작함을 확인하였습니다.

5.2. 주요 구현 내용 및 특징

듀얼 채널 AXI GPIO를 활용하여 스위치 입력과 LED 출력을 효율적으로 처리하였습니다.

128KB 로컬 메모리 제약 조건을 만족시키기 위해 xil_printf 등 표준 라이브러리 사용을 최소화하고, 필수 기능을 수행하는 초간단 자체 함수(Uart_Send_String, Minimal_Atoi, Minimal_Read_Line)를 구현하여 코드 크기를 최적화하였습니다.

SW0 상태를 지속적으로 폴링하여 두 가지 동작 모드를 전환하고, 각 모드에 따른 LED 제어 로직을 구현하였습니다.

UART 통신 시 에코 기능을 제공하고, 유효하지 않은 입력에 대한 기본적인 처리 로직을 포함하였습니다.

5.3. 고찰 및 배운 점

프로젝트 진행 중 겪었던 가장 큰 어려움은 128KB 메모리 제약으로 인한 링크 오류였습니다.

이를 해결하는 과정에서 임베디드 시스템에서 표준 라이브러리 함수가 차지하는 메모리 공간의 크기를 실감하였으며, 제한된 리소스 환경에서의 코드 최적화 기법(라이브러리 의존성 제거, 자체 함수 구현)의 중요성을 체감하였습니다.

또한, Vivado에서 하드웨어 설정(특히 메모리 크기)을 변경했을 때, Vitis 프로젝트에 해당 변경 사항을 올바르게 반영하기 위한 일련의 업데이트 절차(rebuild, re-export, flatform update, build clean)가 매우 중요하다는 것을 경험적으로 학습하였습니다.

이 절차를 누락하면 하드웨어와 소프트웨어 간의 불일치로 인해 예상치 못한 오류(메모리 오버플로우, 오작동 등)가 발생할 수 있음을 알게 되었습니다.

결과적으로 본 프로젝트는 단순한 기능 구현을 넘어, 실제 임베디드 시스템 개발 과정에서 마주칠 수 있는 리소스 제약 문제와 툴체인 활용의 중요성을 깊이 이해하는 계기가 되었습니다.